

```

1  import java.util.ArrayList;
2  import java.util.Arrays;
3  import java.util.Scanner;
4
5  public class Notes {
6      public static void main(String[] args) {
7
8
9
10         //          == PHASE 1: The Basics & Input ==
11
12         // Setting up the Scanner
13         Scanner in = new Scanner(System.in); // Creating scanner
14         object
15         // 1D Array Declaration
16
17         // Syntax A: Definition with size (Empty)
18         int[] arr = new int[5]; // Creates a box of 5 slots. All are 0
19         by default.
20
21         // Syntax B: Definition with values (Populated)
22         int[] nums = {1, 2, 3, 4}; // Creates size 4 and fills it
23         immediately.
24
25         /*
26         FAANG Note: In an interview, if you know data beforehand,
27         always use Syntax B.
28         It is cleaner and faster to write.
29         */
30
31         //          == PHASE 2: Iteration (Looping) ==
32
33         // INPUT
34         // 1. The Standard for Loop
35         for (int i = 0; i < arr.length; i++) {
36             arr[i] = in.nextInt(); // We need 'i' to tell the computer
37             WHERE to put the data
38         }
39
40         // OUTPUT
41         // 2. The Enhanced for Loop (For-Each)
42         for (int num: arr) {
43             System.out.print(num + " ");
44         }
45
46         System.out.println();
47
48         System.out.println(Arrays.toString(arr));
49

```

```

49
50
51     //      == PHASE 4: 2D Arrays (Matrices) ==
52
53     // Syntax
54     int[][] arr2D = new int[3][3]; // 3 rows, 3 columns
55
56     // Input Logic
57     for (int row = 0; row < arr2D.length; row++) {
58         for (int col = 0; col < arr2D.length; col++) {
59             arr2D[row][col] = in.nextInt();
60         }
61     }
62
63
64     //      == PHASE 5: Jagged Arrays ==
65
66     // Syntax
67     int[][] arrJagged = {
68         {1, 2, 4, 4},
69         {5, 6,},
70         {7, 8, 9}
71     };
72
73     // Dynamic Initialisation: Define rows later
74     String[][] arrDynamic = new String[3][]; // Define rows only
75     arrDynamic[0] = new String[4]; // Make row 0 size 4
76     arrDynamic[1] = new String[2]; // Make row 1 size 2
77     arrDynamic[3] = new String[3]; // Make row 2 size 3
78
79     /*
80     FAANG note: This is used in optimisation. If you are storing
81     comments on a post, one post might have
82     1,000 comments and another might have 0. A jagged array saves
83     memory compared to a fixed square matrix.
84     */
85
86
87     //      == PHASE 6: ArrayList (Dynamic Arrays) ==
88
89     // Syntax: Note: You cannot use primitives (int) here. You
90     // must use Wrapper Classes (Integer).
91     ArrayList<Integer> list = new ArrayList<>(5);
92     list.add(23); // Adds to the end
93     list.add(45);
94
95     // Continuous Input: A common pattern to read input the input
96     // stops
97     while (in.hasNextInt()) {
98         list.add(in.nextInt());
99     }

```

```

98
99
100
101
102         //      == PHASE 7: Multidimensional ArrayList ==
103
104         // Syntax
105         ArrayList<ArrayList<Integer>> multiAl = new ArrayList<>();
106
107         // Initialisation (Crucial Step): Unlike a 2D array, you
cannot just say list.get(0).add(1).
108         // You must initialise the inner lists first!
109         for (int i = 0; i < 3; i++) { // Create the empty lists
inside the main list
110             multiAl.add(new ArrayList<>());
111         }
112
113         // Now you can add data
114         multiAl.get(0).add(10); // Go to list 0, add 10
115
116         /*
117         FAANG Note: This Structure is exactly how Adjacency Lists are
built. An Adjacency List is the
118         primary way we represent Graphs (like connections in LinkedIn
or Facebook). You will use this
119         structure constantly in advanced interviews.
120         */
121     }
122
123
124
125
126
127         //      == PHASE 3: Memory & References (Critical) ==
128
129         static void change (int[] arr) {
130             arr[0] = 99; // This modifies the ORIGINAL array in main
()!!!!!!!!!!
131         }
132
133
134
135     }
136

```